

Pop-up ALV

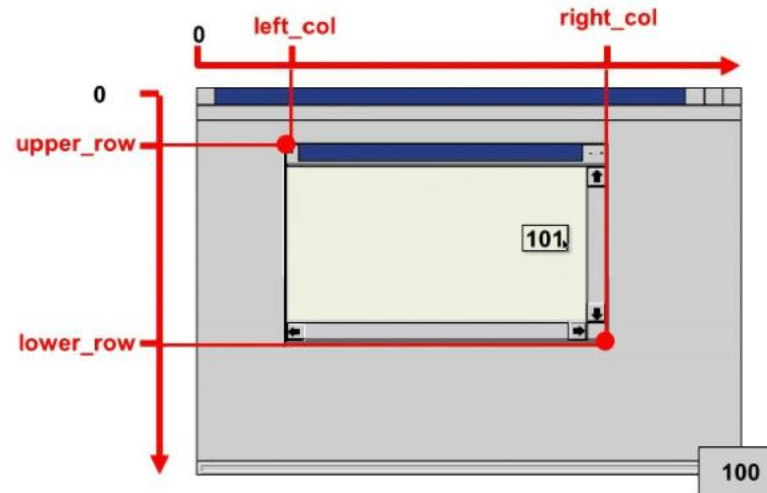
- `cl_gui_custom_container` 클래스를 참조하여 객체 선언,
- 메인 스크린은 화면 번호가 100씩, 팝업 스크린은 10씩, 서브 스크린은 1씩 증가한다.

TAW10

Figure 153: Window Coordinates

Unit 11 Lesson 6

```
CALL SCREEN 101  
  STARTING AT left_col upper_row  
  ENDING   AT right_col lower_row.
```



This material is for the exclusive use of SAP Trainer

SAP Memory & ABAP Memory

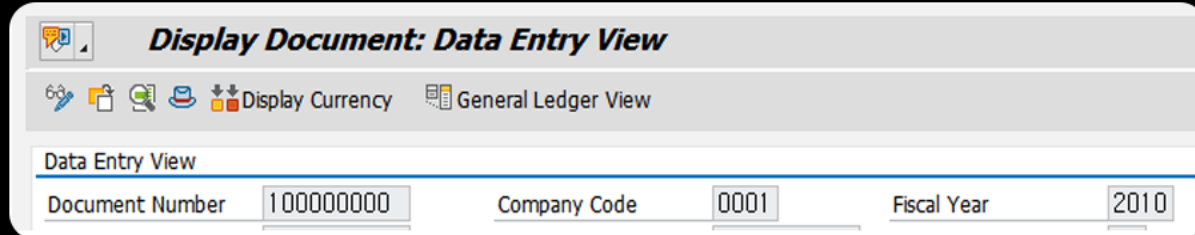
- ABAP Memory : 같은 내부 세션 내에서 데이터를 공유하는 메모리 공간 (프로그램 A $\iff$ B), 대규모 데이터
- SAP Memory : 트랜잭션 간 데이터를 공유하는 글로벌 메모리 공간, 메모리 ID 필요 (프로그램 $\Rightarrow$ FB03), 소규모

```
REPORT zprogram1.  
DATA lt_data TYPE TABLE OF string.  
  
APPEND 'Data 1' TO lt_data.  
APPEND 'Data 2' TO lt_data.  
  
EXPORT data = lt_data TO MEMORY.
```

```
REPORT zprogram2.  
  
DATA lt_data TYPE TABLE OF string.  
  
IMPORT data = lt_data FROM MEMORY.
```

```
SET PARAMETER ID : 'BLN' FIELD gs_itab-belnr,  
                  'BUK' FIELD gs_itab-bukrs,  
                  'GJR' FIELD gs_itab-gjahr.
```

```
"FB03 (Display FI Document) 트랜잭션 호출  
CALL TRANSACTION 'FB03' AND SKIP FIRST SCREEN.
```



The screenshot shows the SAP 'Display Document: Data Entry View' interface. At the top, there's a title bar with the text 'Display Document: Data Entry View'. Below it, there are several icons and labels: 'Display Currency' and 'General Ledger View'. The main area is titled 'Data Entry View' and contains three input fields: 'Document Number' with the value '100000000', 'Company Code' with the value '0001', and 'Fiscal Year' with the value '2010'.

Data Entry View		
Document Number	100000000	Company Code
		0001
		Fiscal Year
		2010

ALV 컬러 지정

- `lvc_t_scol` `lvc_s_scol` 을 사용하여 각 Cell의 컬러를 지정할 수 있음
- ALV 출력 대상인 인터널 테이블 구조에 `lvc_t_scol` 을 담는다.
- LOOP 문과 `lvc_s_scol` 를 이용하여 한 줄마다 어떤 필드에 색을 지정할 지 할당한 후 itab을 MODIFY, Layout에 지정

```
DATA : BEGIN OF gs_data,  
        ~  
        color      TYPE lvc_t_scol,  
END OF gs_data,
```

```
LOOP AT gt_itab INTO gs_itab.  
    lv_idx = sy-tabix.  
    CLEAR ls_scol.  
    CASE gs_itab-carriid.  
        WHEN 'LH'.  
            ls_scol-fname = 'COUNTRYFR'.  
            ls_scol-color-col = 3.  
            APPEND ls_scol TO gs_itab-color.  
        ENDCASE.  
    MODIFY gt_itab FROM gs_itab INDEX lv_idx TRANSPORTING color.  
ENDLOOP.  
gs_layout-ctab_fname = 'COLOR'. “마지막으로 layout에 지정
```

ALV 컬러 지정

GT_SPFLI						
CARRID	CONNID	COUNTRYFR	CITYFROM	COUNTRYTO	CITYTO	COLOR
DL	0106	US	NEW YORK	DE	FRANKFURT	FNAME COLOR NOKEYCOL
DL	1699	US	NEW YORK	US	SAN FRANCISCO	FNAME COLOR NOKEYCOL
DL	1984	US	SAN FRANCISCO	US	NEW YORK	FNAME COLOR NOKEYCOL
JL	0407	JP	TOKYO	DE	FRANKFURT	FNAME COLOR NOKEYCOL
						COL INT INV X
						6 0 0
JL	0408	DE	FRANKFURT	JP	TOKYO	FNAME COLOR NOKEYCOL
						COL INT INV X
						6 0 0
LH	0400	DE	FRANKFURT	US	NEW YORK	FNAME COLOR NOKEYCOL
						COUNTRYFR COL INT INV
						3 0 0
LH	0401	US	NEW YORK	DE	FRANKFURT	FNAME COLOR NOKEYCOL
						COUNTRYFR COL INT INV
						3 0 0
LH	0402	DE	FRANKFURT	US	NEW YORK	FNAME COLOR NOKEYCOL
						COUNTRYFR COL INT INV
						3 0 0

ALV 컬러 지정

LVC_T_SCOL (Table Type: Container for Color Settings)



LVC_S_SCOL (Structure: Single Field Color Setting)

FNAME
(Field Name)

COLOR
(Refers to LVC_S_COLO)

NOKEYCOL
(Ignore Key Color)



LVC_S_COLO (Structure: Color Definition)

COL
(Color Index)
- Values: 1~7

INT
(Intensified)
- Values: 0 or 1

INV
(Inverse)
- Values: 0 or 1



DML (Data Manipulation Language)

- DML은 데이터 조작어로 **SELECT, INSERT(삽입), UPDATE(갱신), MODIFY(수정), DELETE(삭제)**가 있다.
- **인터널 테이블**을 대상으로 사용하는 DML과 **DB 테이블**을 대상으로 사용하는 DML의 문법이 다소 다름
- DDL(데이터 정의어), DCL(데이터 제어어)도 있지만 이는 ABAP Program이 아닌 ABAP Dictionary의 영역
- **DB 테이블과 인터널 테이블의 데이터 타입과 구조(필드 개수 등)는 완벽히 일치해야 함**
- 지역변수 선언과 MOVE CORRESPONDING을 사용하는 것을 강력히 권장

```
DATA : lt_save TYPE TABLE OF ZC2T260006.
```

"gt_data가 lt_save보다 필드가 1개 많을 경우

```
MOVE-CORRESPONDING gt_data TO lt_save.
```

INSERT

* DB 테이블: INSERT 문 (단일 행, 다중 행)

```
INSERT INTO scarr VALUES ls_wa.
```

```
INSERT scarr FROM TABLE lt_itab.
```

* Internal 테이블: INSERT, APPEND (단일 행, 다중 행)

```
INSERT ls_wa INTO lt_itab INDEX 1.
```

```
APPEND ls_wa TO lt_itab.
```

UPDATE

* DB 테이블: UPDATE 문

```
UPDATE scarr SET carrname = 'New Name' WHERE carrid = 'AA'.
```

```
UPDATE scarr FROM ls_wa.
```

```
UPDATE scarr FROM TABLE lt_itab.
```

* Internal 테이블: MODIFY

```
MODIFY TABLE lt_itab FROM ls_wa.
```

```
MODIFY lt_itab FROM ls_wa INDEX 1.
```

```
MODIFY lt_itab FROM ls_wa TRANSPORTING carrname WHERE carrid = 'AA'.
```


UPDATE

* DB 테이블: DELETE 문

```
DELETE FROM scarr WHERE carrid = 'AA'.
```

```
DELETE scarr FROM ls_wa.
```

```
DELETE scarr FROM TABLE lt_itab.
```

* Internal 테이블: DELETE

```
DELETE TABLE lt_itab WITH TABLE KEY carrid = 'AA'.
```

```
DELETE lt_itab INDEX 1.
```

```
DELETE lt_itab WHERE carrid = 'AA'.
```

UPDATE

* DB 테이블: MODIFY 문 (존재하면 UPDATE, 없으면 INSERT)

```
MODIFY scarr FROM ls_wa.
```

```
MODIFY scarr FROM TABLE lt_itab.
```

* Internal 테이블: MODIFY (항상 기존 레코드 수정, INSERT 기능 없음)

```
MODIFY TABLE lt_itab FROM ls_wa.
```

MODIFY

* DB 테이블: MODIFY 문 (존재하면 UPDATE, 없으면 INSERT)

```
MODIFY scarr FROM ls_wa.
```

```
MODIFY scarr FROM TABLE lt_itab.
```

* Internal 테이블: MODIFY (항상 기존 레코드 수정, INSERT 기능 없음)

```
MODIFY TABLE lt_itab FROM ls_wa.
```

TCL(Transaction Control Language)

- 트랜잭션 제어어라고 하며, COMMIT WORK 와 ROLLBACK WORK 가 있음
 - COMMIT WORK는 모든 DB 변경을 영구적으로 저장하고, ROLLBACK WORK는 모든 변경사항을 취소함.
- 프로그램이 정상적으로 종료되면 자동으로 COMMIT 되고, 오류 발생 시 자동으로 ROLLBACK 됨
 - 그러나 명시적으로 COMMIT 및 ROLLBACK을 선언하고 제어해주는 것을 권장
 - COMMIT의 경우도 COMMIT WORK AND WAIT을 같이 사용함으로써 커밋이 끝날 때까지 대기
- Function Module 내부에서는 절대 COMMIT을 선언하지 않아야 함
 - FM 호출 이전에 호출자가 수행했던 DB 작업이 있었다면 FM에서 COMMIT이 되버림
 - 이후 호출자에서 DB 작업을 추가로 수행한 후 ROLLBACK 하더라도 이전에 COMMIT 된 작업은 롤백 안됨

ALV 내용 수정 가능하게 하기

- ALV Color와 거의 동일한 방식으로 작동됨 (itab 구조 정의, LOOP문 순회, Layout 및 필드 카탈로그 설정)
- `lvc_s_styl` `lvc_t_styl` 이란 ALV 스타일 테이블/구조를 사용하여 구현할 수 있으며, itab 구조 내부에 포함해야 함

```
DATA: BEGIN OF gs_data.  
      INCLUDE STRUCTURE ztcart_20_01.  
DATA:  celltab TYPE lvc_t_styl, * celltab이란 이름으로 정의  
      END OF gs_data,  
      gt_data LIKE TABLE OF gs_data,
```

- LOOP 문으로 itab을 한 줄씩 순회하면서 수정 가능하게 만들 필드와 읽기 전용으로 만들 필드를 지정
- `gs_layout-stylefname = 'CELLTAB'` 과 `gs_fcat-edit = 'X'` 으로 활성화 (레이아웃/필드 카탈로그 지정)

ALV 내용 수정 가능하게 하기

```
FORM screen_change .  
    DATA : ls_style TYPE lvc_s_styl,  
            lv_idx   TYPE sy-tabix.  
  
    LOOP AT gt_data INTO gs_data.  
        lv_idx = sy-tabix.  
  
        CLEAR : ls_style, gs_data-celltab.  
  
        PERFORM set_style USING 'SHOP' ''.  
        PERFORM set_style USING 'CUSTOM_ID' ''.  
        PERFORM set_style USING 'MAKTX' ''.  
        PERFORM set_style USING '' 'X'.  
    ENDLOOP.  
ENDFORM.
```

```
FORM set_style USING pv_fieldName pv_flag.  
    DATA : ls_style TYPE lvc_s_styl.  
    ls_style-fieldname = pv_fieldName.  
  
    CASE pv_flag.  
        WHEN 'X'.  
            ls_style-style =  
                cl_gui_alv_grid⇒mc_style_enabled.  
        WHEN ''.  
            ls_style-style =  
                cl_gui_alv_grid⇒mc_style_disabled.  
    ENDCASE.  
  
    INSERT ls_style INTO TABLE gs_data-celltab.  
    CLEAR ls_style.  
ENDFORM.
```

ALV 내용 수정 가능하게 하기

```
FORM set_layout .  
    gs_layout-zebra      = 'X'.  
    ~  
    gs_layout-stylefname = 'CELLTAB'.  
ENDFORM.
```

ALV 내용 수정 가능하게 하기

```
FORM register_event .  
    "ALV Grid의 셀 값을 수정할 때마다 즉시 이벤트를 발생시켜서  
    "해당 변경사항을 실시간으로 처리(수량 필드 변경 시 금액 바로 재계산 등)  
    CALL METHOD go_alv_grid→register_edit_event  
        EXPORTING  
            i_event_id = cl_gui_alv_grid⇒mc_evt_modified.  
  
    "ALV 전체 수정 혹은 조회 상태로 변경  
    CALL METHOD go_alv_grid→set_ready_for_input  
        EXPORTING  
            i_ready_for_input = 1. " 1 : Editable , 0 : Read-Only  
ENDFORM.
```


ALV 내용 수정 가능하게 하기

```
FORM toggle_edit_mode .  
  CASE gv_okcode.  
    WHEN 'TOGL'.  
      CASE gv_flag.  
        WHEN '0'.  
          CALL METHOD go_alv_grid→set_ready_for_input  
            EXPORTING  
              i_ready_for_input = 1. " 1 : Editable , 0 : Read-Only  
          gv_flag = '1'.  
        WHEN '1'.  
          CALL METHOD go_alv_grid→set_ready_for_input  
            EXPORTING  
              i_ready_for_input = 0. " 1 : Editable , 0 : Read-Only  
          gv_flag = '0'.  
      ENDCASE.  
    ENDCASE.  
  ENDFORM.
```

읽기 전용이면 편집 모드로

편집 모드면 읽기 전용으로

ALV Toolbar에서 버튼 숨기기

- ALV Color, ALV Style와 또 동일한 방식으로 작동됨 (전역변수 선언, 테이블 구조에 추가, ALV 출력 메소드에 지정)
- `ui_functions` 라는 테이블 구조를 사용하여 숨길 버튼을 정할 수 있음

```
FORM toolbar_exclude .
```

```
  CLEAR gt_toolbar.
```

```
  *-- 숨길 버튼을 APPEND 하여 추가
```

```
  APPEND : cl_gui_alv_grid⇒mc_fc_loc_insert_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_delete_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_append_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_copy_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_move_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_delete_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_append_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_copy_row TO gt_toolbar,
```

```
          cl_gui_alv_grid⇒mc_fc_loc_move_row TO gt_toolbar,
```

```
ENDFORM.
```

```
DATA : gt_toolbar TYPE ui_functions.
```

```
CALL METHOD go_alv_grid→set_table_for_first_display
```

```
  EXPORTING
```

```
    is_variant          = gs_variant
```

```
    ...
```

```
    it_toolbar_excluding = gt_toolbar
```

```
  CHANGING
```

```
    ...
```

"여기가 핵심

ALV 정리 (26. 01. 07 23:53)

- 1. 테이블 SELECT 후 ALV 출력 (텍스트 테이블이 있을 경우 READ TABLE)
- 2. 저장 버튼 / 토글버튼 / 행추가 / 행삭제 버튼 추가 (MENU BAR 혹은 Application Bar에 추가)
- 3. screen_change 서브루틴 구현하여 특정 필드 수정/읽기 전용 구현 (lvc_t_styl, lvc_s_styl 사용)
- 4. PAI 모듈에 USER_COMMAND_0100. 모듈 구현
- 5. gv_okcode에 따라 분기하여 저장 버튼 / 토글 버튼 / 행추가 / 행삭제로 분기
- 6. 토글 버튼은 `CALL METHOD go\_alv\_grid→set\_ready\_for\_input` 메소드 호출하여 구현
- 7. 행 삽입 / 삭제 버튼은 `CALL METHOD go\_alv\_grid→get\_selected\_rows` 메소드 호출하여 구현
 - 행 삭제는 인덱스 고임 방지를 위하여 내림차순으로 정렬 후 뒤에서부터 삭제해야 함
 - 삭제할 행은 READ TABLE로 읽어들이고 후 삭제한 행을 담은 임시 테이블에 저장하여야 한다.
 - DML문이 끝나면 screen_change 서브루틴 및 refresh_screen 서브루틴 호출하여 변경점 반영
- 8. 저장 버튼은 `CALL METHOD go\_alv\_grid→check\_changed\_data` 호출하여 ALV의 변경점을 itab에 반영
 - 그 후 임시 저장한 테이블을 활용하여 DELETE문으로 삭제할 행은 삭제한 후 MODIFY문으로 변경/추가
- 9. Toolbar의 버튼을 숨기거나(gt_toolbar) EVENT 메소드(register_edit_event) 호출 등이 필요한 경우 별도 선언